

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»

Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчёт защищён с оценкой
Преподаватель С.В. Умбетов
«___» _____ 2026 г.

Отчёт по лабораторной работе №1

Студент группы 1ИИС-41 _____ В.А. Помешкина
группа ф.и.о.

Преподаватель старший преподаватель, доцент С.В. Умбетов

должность, учёная степень

ф.и.о.,

Барнаул 2026

Лабораторная работа №2 работа с табличными данными в Pandas

Цель работы: Освоить базовые и продвинутые приёмы работы с табличными данными в библиотеке Pandas на примере реального набора данных о пассажирах «Титаника». Научиться загружать, исследовать, фильтровать, агрегировать и преобразовывать табличные данные, а также делать аналитические выводы на основе полученных результатов.

Задачи работы: В ходе выполнения лабораторной работы необходимо:

1. Изучить теоретические основы работы с библиотекой Pandas: объекты Series и DataFrame , их создание и основные свойства.
2. Освоить чтение данных из CSV-файлов с помощью функции `pd.read_csv()` и её параметров.
3. Научиться получать первичную информацию о датафрейме с помощью методов `.head()` , `.tail()` , `.info()` , `.describe()` .
4. Освоить индексацию и доступ к данным через `.loc` , `.iloc` и обращение к столбцам.
5. Научиться фильтровать данные по одному и нескольким условиям с использованием логических операторов.
6. Освоить статистические методы (`.mean()` , `.max()` , `.min()` , `.sum()` , `.std()` , `.count()`) и их применение к датафреймам и отдельным столбцам.
7. Изучить группировку данных с помощью `.groupby()` и агрегацию через `.agg()` .
8. Научиться строить сводные таблицы с помощью `.pivot_table()` .
9. Освоить работу с пропущенными значениями: обнаружение, удаление, заполнение.
10. Научиться создавать новые признаки на основе существующих с использованием арифметических операций, `pd.cut()` и `.apply()` .
11. Провести комплексный аналитический анализ датасета Titanic и сформулировать содержательные выводы.
12. Закрепить полученные навыки ответами на контрольные вопросы.

Задание принял: _____ Помешкина В.А. _____

Задание 1

Загрузка и первичный осмотр данных

import pandas as pd
titanic = pd.read_csv('train.csv')
display(titanic.head(10))
display(titanic.tail(5))

***	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

Рисунок 1 – вывели первые 10 строк таблицы с помощью .head(10)

886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Рисунок 2 – вывели 5 строк с помощью .tail()

Задание 2

Получение информации о датафрейме

```
import pandas as pd
titanic = pd.read_csv('train.csv')
titanic.info()
display(titanic.describe())
display(titanic.describe(include=['object']))
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype  
---  -
 0   PassengerId   891 non-null    int64  
 1   Survived      891 non-null    int64  
 2   Pclass        891 non-null    int64  
 3   Name          891 non-null    object  
 4   Sex           891 non-null    object  
 5   Age           714 non-null    float64 
 6   SibSp         891 non-null    int64  
 7   Parch         891 non-null    int64  
 8   Ticket        891 non-null    object  
 9   Fare          891 non-null    float64 
10   Cabin         204 non-null    object  
11   Embarked      889 non-null    object  
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Рисунок 3 – использовали метод .info() для получения информации о типах данных и количестве непустых значений

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Рисунок 4 – применили метод .describe() к числовым столбцам.

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Dooley, Mr. Patrick	male	347082	G6	S
freq	1	577	7	4	644

Рисунок 5 – применили метод .describe(include=['object']) к текстовым столбцам

Задание 3

Работа с Series

```

import pandas as pd
titanic = pd.read_csv('train.csv')
embarked_series = titanic['Embarked']
first_5 = embarked_series.head(5).copy()
first_5.index = ['Пассажир_1', 'Пассажир_2', 'Пассажир_3', 'Пассажир_4', 'Пассажир_5']
display(first_5)
display(first_5.loc['Пассажир_2'])
display(first_5.iloc[1])

```

...	Embarked
Пассажир_1	S
Пассажир_2	C
Пассажир_3	S
Пассажир_4	S
Пассажир_5	S

dtype: object
'C'
'C'

Рисунок 6 - извлекаем столбец Embarked в Series, задаем первому 5-элементному срезу индексы Пассажир_1–Пассажир_5 и берём элемент через .loc['Пассажир_2'] и .iloc[1].

Задание 4

Статистические методы

```
import pandas as pd

titanic = pd.read_csv('train.csv')

mean_age = titanic['Age'].mean()
max_fare = titanic['Fare'].max()
min_fare = titanic['Fare'].min()
std_age = titanic['Age'].std()

print(f"Средний возраст: {mean_age:.2f}")
print(f"Максимальная стоимость: {max_fare}, Минимальная стоимость: {min_fare}")
print(f"Стандартное отклонение возраста: {std_age:.2f}")

... Средний возраст: 29.70
Максимальная стоимость: 512.3292, Минимальная стоимость: 0.0
Стандартное отклонение возраста: 14.53
```

Рисунок 7 - считаем средний возраст через `.mean()`, находим крайние цены билетов с помощью `.max()` и `.min()`, и вычисляем разброс возраста через `.std()`.

Задание 5

Фильтрация данных по условиям

```
import pandas as pd
titanic = pd.read_csv('train.csv')
over_30 = titanic[titanic['Age'] > 30]
above_avg_fare = titanic[titanic['Fare'] > titanic['Fare'].mean()]
female_1st = titanic[(titanic['Pclass'] == 1) & (titanic['Sex'] == 'female')]
avg_age_female_1st = female_1st['Age'].mean()
print(f"пассажиров старше 30 лет: {len(over_30)}")
print(f"пассажиров с билетом выше среднего: {len(above_avg_fare)}")
print(f"женщин в 1 классе: {len(female_1st)}")
print(f"средний возраст женщин 1 класса: {avg_age_female_1st:.2f}")

... пассажиров старше 30 лет: 305
пассажиров с билетом выше среднего: 211
женщин в 1 классе: 94
средний возраст женщин 1 класса: 34.61
```

Рисунок 8 - фильтруем строки по условиям через квадратные скобки [...], комбинируем условия `Pclass == 1` и `Sex == 'female'` через оператор `&`, и считаем их средний возраст методом `.mean()`

Задание 6

Комбинированная фильтрация и статистика

```
import pandas as pd
titanic = pd.read_csv('train.csv')
survived_under_18 = titanic[(titanic['Survived'] == 1) & (titanic['Age'] < 18)]
avg_fare_survived_under_18 = survived_under_18['Fare'].mean()
count_survived_under_18 = len(survived_under_18)
print(f"выживших моложе 18 лет: {count_survived_under_18}")
print(f"средняя стоимость их билета: {avg_fare_survived_under_18:.2f}")
```

... выживших моложе 18 лет: 61
средняя стоимость их билета: 34.86

Рисунок 9 – выживших детей фильтруем через `(titanic['Survived'] == 1) & (titanic['Age'] < 18)` и считаем кол-во пассажиров через `len()` и среднюю стоимость их билета методом `.mean()`

Задание 7

Группировка данных

```
import pandas as pd
titanic = pd.read_csv('train.csv')
pclass_grouped = titanic.groupby('Pclass').agg(
    mean_age=('Age', 'mean'),
    count_passengers=('PassengerId', 'count'),
    mean_fare=('Fare', 'mean')
)
pclass_grouped_sorted = pclass_grouped.sort_values(by='mean_fare', ascending=False)
display(pclass_grouped_sorted)
```

... mean_age count_passengers mean_fare

Pclass	mean_age	count_passengers	mean_fare
1	38.233441	216	84.154687
2	29.877630	184	20.662183
3	25.140620	491	13.675550

Рисунок 10 - группируем пассажиров по Pclass с помощью `.groupby()`, считем через `.agg()` средний возраст, кол-во человек и средний чек, после чего сортируем таблицу `mean_fare` по убыванию `.sort_values(ascending=False)`.

Задание 8

Сводная таблица

```
import pandas as pd
titanic = pd.read_csv('train.csv')
pivot_survived = titanic.pivot_table(
    values='Survived',
    index='Pclass',
    columns='Sex',
    aggfunc='mean'
)

display(pivot_survived)
```

	Sex	female	male
Pclass			
1		0.968085	0.368852
2		0.921053	0.157407
3		0.500000	0.135447

Рисунок 11 - создаем сводную таблицу через `pd.pivot_table(titanic, values='Survived', index='Pclass', columns='Sex', aggfunc='mean')`, строки — классы, столбцы — пол, а значения средняя доля выживших

Задание 9

Работа с пропущенными значениями

```
[2]
0 сек.
import pandas as pd
titanic = pd.read_csv('train.csv')

print('Пропуски (количество):')
print(titanic.isnull().sum())

print('\nПропуски (%):')
print(titanic.isnull().sum() / len(titanic) * 100)

titanic['Embarked'] = titanic['Embarked'].fillna(titanic['Embarked'].mode()[0])
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].median())

print('\nПропуски после заполнения:')
print(titanic[['Age', 'Embarked']].isnull().sum())

print('\nДубликатов по PassengerId:', titanic.duplicated(subset=['PassengerId']).sum())
titanic = titanic.drop_duplicates(subset=['PassengerId'], keep='first')
print('Дубликатов после удаления:', titanic.duplicated(subset=['PassengerId']).sum())
```

Пропуски (количество):	
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687

Рисунок 12 - считаем кол-во и % пропусков по столбцам, заполняем Embarked самым частым значением и Age медианой, проверяем и удаляем дубликаты по PassengerId

Задание 10

Создание новых признаков и расширенный анализ

```
4] 0 сек.
import pandas as pd
titanic = pd.read_csv('train.csv')

titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(
    titanic['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['ребёнок', 'подросток', 'молодой', 'взрослый', 'пожилой']
)
age_stats = titanic.groupby('AgeGroup', observed=True).agg(
    survived_rate=('Survived', 'mean'),
    mean_fare=('Fare', 'mean'),
    count=('Survived', 'count')
)
display(age_stats)
pivot_age = titanic.pivot_table(
    values='Survived',
    index='AgeGroup',
    aggfunc='mean',
    observed=True
)
display(pivot_age)
```

Рисунок 13 – создаём признаки FamilySize, IsAlone и AgeGroup через pd.cut. по возрастным группам считаем долю выживших, среднюю цену и кол-во

...	survived_rate	mean_fare	count	
AgeGroup				
ребёнок	0.579710	31.536352	69	
подросток	0.428571	33.451313	70	
молодой	0.382682	29.925929	358	
взрослый	0.400000	44.259658	195	
пожилой	0.227273	41.371214	22	
Survived				
AgeGroup				
ребёнок	0.579710			
подросток	0.428571			
молодой	0.382682			
взрослый	0.400000			
пожилой	0.227273			

Рисунок 14 - строим сводную таблицу и делаем вывод

Задание 11

Комплексный аналитический отчёт

```
import pandas as pd
titanic = pd.read_csv('train.csv')
print('Топ-3 порта по количеству пассажиров:')
print(titanic['Embarked'].value_counts().head(3))

port_stats = titanic.groupby('Embarked').agg(
    mean_age=('Age', 'mean'),
    mean_fare=('Fare', 'mean'),
    survived_rate=('Survived', 'mean'),
    first_class_share=('Pclass', lambda x: (x == 1).mean())
)
display(port_stats)
```

... Топ-3 порта по количеству пассажиров:

Embarked

S 644
C 168
Q 77
Name: count, dtype: int64

	mean_age	mean_fare	survived_rate	first_class_share
Embarked				
C	30.814769	59.954144	0.553571	0.505952
Q	28.089286	13.276030	0.389610	0.025974
S	29.445397	27.079812	0.336957	0.197205

Рисунок 15 - топ-3 порта по числу пассажиров, для каждого — средний возраст, средняя цена, доля выживших и доля 1 класса; вывод о порте с наибольшей выживаемостью

Задание 12

Сравнительный анализ по размеру семьи

```
import pandas as pd
titanic = pd.read_csv('train.csv')
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1

family_stats = titanic.groupby('FamilySize').agg(
    survived_rate=('Survived', 'mean'),
    mean_age=('Age', 'mean'),
    mean_fare=('Fare', 'mean'),
    count=('Survived', 'count')
)
family_stats = family_stats[family_stats['count'] > 10]
family_stats_sorted = family_stats.sort_values('survived_rate', ascending=False)
display(family_stats_sorted)
```

... survived_rate mean_age mean_fare count

FamilySize

4	0.724138	18.274815	54.864510	29
3	0.578431	26.035806	39.692482	102
2	0.552795	31.391511	49.894129	161
7	0.333333	15.166667	29.366667	12
1	0.303538	32.220297	21.242689	537
5	0.200000	20.818182	58.094453	15
6	0.136364	18.409091	73.722727	22

Рисунок 16 - группируем по FamilySize и считаем долю выживших, средний возраст, среднюю цену и кол-во; фильтруем семьи >10 пассажиров, сортируем по доле выживших и делаем вывод, где она максимальна

https://colab.research.google.com/drive/1k523WNIso-HT_jVmGQVjxW-ydPzPNt4t?usp=sharing

Вывод:

В ходе лабораторной работы я освоила приёмы работы с табличными данными в Pandas на примере датасета «Титаник». Научилась загружать данные из CSV, получать информацию о датафрейме, работать с Series и DataFrame, индексировать через loc и iloc, фильтровать по одному и нескольким условиям, применять статистические методы, группировать данные через groupby с агрегацией agg и строить сводные таблицы pivot_table. Также освоила обработку пропусков, удаление дубликатов и создание новых признаков через pd.cut и apply. В результате анализа выяснила, что чаще выживали дети, женщины и пассажиры первого класса.